React for Java Backend Developers

As a Java backend developer, you're used to working with frameworks like Spring Boot, Hibernate, and REST APIs. React is the leading JavaScript library for building interactive frontends that consume those backend APIs. This guide explains React concepts with a focus on how they relate to Java backend development.

1. Core React Concepts (Java Developer’s Perspective)

React is a library for building **component-based user interfaces**. Think of React components like Java classes, but focused on UI rather than business logic. Each component manages its own state and renders HTML dynamically.

1. **Component = Class**: Similar to Java classes, components encapsulate logic and structure.
2. **Props = Method Parameters**: Props are inputs passed to components, similar to arguments in Java methods.
3. **State = Object Field**: State represents mutable data specific to a component.
4. **Render = toString()**: Defines how a component visually represents itself.

2. Connecting React with a Java (Spring Boot) Backend

React typically communicates with your Java backend through REST or GraphQL APIs. The React frontend runs separately (port 3000), while your Spring Boot backend runs on another port (e.g., 8080). You connect them using HTTP requests.

1. **Spring Boot**: Exposes REST APIs using @RestController.
2. **React**: Uses fetch() or axios to call those APIs.
3. **CORS**: Must be configured in Spring Boot to allow frontend access.

3. Example: React Fetching Data from Spring Boot

Spring Boot (Backend):
@RestController @RequestMapping("/api") public class EmployeeController { @GetMapping("/employees") public List getAllEmployees() { return employeeService.findAll(); } }
React (Frontend):
import { useEffect, useState } from "react"; function EmployeeList() { const [employees, setEmployees] = useState([]); useEffect(() => { fetch("http://localhost:8080/api/employees") .then(res => res.json()) .then(data => setEmployees(data)); }, []); return ({employees.map(emp => ({emp.name}))}); } export default EmployeeList;

4. React Ecosystem for Java Developers

1. **React Router**: Like @RequestMapping, but for client-side navigation.
2. **Redux / Context API**: Like Spring Beans managing shared state.
3. **Hooks**: Similar to lifecycle methods or callbacks in Java.
4. **Axios**: HTTP client like RestTemplate or WebClient.

5. Build and Deployment

React apps are built using Node.js. Once built (`npm run build`), you can deploy the static files in your Spring Boot project under `/src/main/resources/static` — allowing Spring Boot to serve the React UI and REST APIs together.

6. Key Takeaways

1. React = UI layer; Spring Boot = API layer.
2. Think of React components like Java classes managing view logic.
3. APIs connect frontend and backend cleanly using JSON.
4. CORS and JSON serialization are key integration points.

Happy Learning! ■